

VIEWS

- A view is a virtual table. A view consists of rows and columns like a table.
- We can create a view by selecting fields from one or more tables present in the database. But it does not contain any data. At run time it contains the data from table and after execution it gets free.
- If data is changing in the original table, the same change is reflected in the view. A view can be built on top of a single table or multiple tables & another view.
- View requires no memory to store data but some memory required for storing definition of view.
- View is defined on which table this table is called as Base table.
- If view is already exist the View can be replaced using 'replace' keyword.
- We can create forcefully view on without existing the table.
- View only allows a user to retrieve data.

Types of view:

▪ **Simple view :**

- 1) The simple view is created from only one single table.
- 2) A simple view also does not have any groups of data.
- 3) Simple view will not have any expression or group function.
- 4) All DML allow in simple views.
- 5) If table drop then view will not drop but view become invalid. View become valid when table again created with same as dropped table column name.

▪ **Complex view:**

- 1) The complex view is created from than one table and also has group function.
- 2) A complex view contains groups of data.
- 3) DML are not allowed.
- 4) Through view we can insert data into child table not in parent table.

Why Views?

- Provides additional level of security by restricting access to a fixed set of rows and columns of a table.
- Hide the data complexity.
- Ease of use and Space savings
- To provide data independence
- To present different views of the same data

• **Creating Views**

Suppose we have EMP and DEPT table. We want to see emp details i.e. department names where they are working. We have to write a long join query.

```
select e.eno,e.ename,e.sal,e.dno,d.dname,d.loc From emp e, dept d where e.dno=d.dno;
```

Instead of writing this join query again and again, we can create a view on these tables by using a CREATE VIEW command:

```
create view vw_empdept as select e.empno, e.ename, e.sal, e.deptno, d.dname, d.loc from emp e, dept d where e.deptno=d.deptno;
```

Now to see the employee details and department names:

```
select * from vw_empdept; // This will show same result as join query.
```

- **View with Check Option Constraint**

Creates view with the constraint that apply on INSERT and UPDATE, DELETE statements.

Example-suppose all the employee working in Department No. 10 belongs to accounts department.

```
create view vw_accounts_staff as select eno, ename, dno from emp where dno=10 with check option constraint ica_accounts_cnst;
```

```
select * from vw_accounts_staff;
```

```
select sum(sal) from vw_accounts_staff;  
select max(sal) from vw_accounts_staff;
```

In “vw_accounts_staff” view you can insert only rows in these departments number 10.

INSERT statement successfully inserts a row into the EMP table through the vw_accounts_staff view: insert into vw_accounts_staff values (110, 'ASHI', 10);

If you insert a row with department number 30, this INSERT statement is rolled back and raise error because with check constraints apply on vw_accounts_staff view it allow only dept number 10:

```
insert into vw_accounts_staff values(111, 'SAMI', 30);
```

- **Creating View without Having the Base Table**

You can create a view without base table but once the base table was created then the view is validated.

```
Create force view vw_stud as select * from student;
```

- **Replacing/Altering Views**

- A view can be drop or recreate. When a view is dropped, all grants of corresponding view rights are revoked from users. After the view is re-created, necessary rights must be regranted.
- A view can be replaced or redefining by CREATE OR REPLACE VIEW statement. This option replaces the current definition of a view, but preserves the present security authorizations.
- If base table was dropped then we will get the message like “view has errors”.
- If base table has been altered but still the view was with the initial definition, we have to replace the view to affect the changes.

```
create or replace view vw_accounts_staff as select empno, ename, deptno from emp where deptno = 30 with check option constraint ica_accounts_cnst;
```

INSERT statement successfully inserts a row into the EMP table through the accounts_staff view because we are change view definition:

```
insert into vw_accounts_staff values (199, 'ABID', 30);
```

- **Dropping Views :** Remove view from database

Example: drop view vw_accounts_staff;

- **Views without DML**

- **Read only view:** Create view vw_dept as select * from dept with read only;
- **View with aggregate functions & group by clause:**
Create view vw_sumsal as select dno, sum(sal) t_sal from emp group by dno;
- **View with distinct:** Create view vw_stud as select distinct no,name from student;

- **Updating Views with DML**

EMP base table that cause the view changes and employee 109's name changes from ASHI to SHRI in the EMP table.

```
update vw_empdept set ename='shri' where eno=109;
```

- **Creating View Join (complex view)**

To provide users with convenient access to information from multiple tables, create a complex view.

```
create view vw_emploc as select e.eno, e.ename, e.dno, e.sal, d.dname, d.loc from emp e, dept d where e.dno=d.dno and d.loc in ('HYD', 'BOM', 'DEL');
```

- **Listing Information about VIEWS**

To see how many views are there in your database:

```
select * from user_views;
```

```
select * from user_updatable_columns where table_name='vw_empdept';
```

☉ Advantages of View

- View provides data security for the same base tables.
- It allows different users to view the same data in different ways at the same time.
- It is used to represent additional information like derived columns.
- It is used to hide complex queries.
- It presents a consistent, unchanged image of the database structure, even if the tables are split or renamed.
- It does not allow direct access to the tables of the data dictionary.

☉ Disadvantages of View

- We cannot use DML operations on View, if there is more than one table.
- When table is dropped view becomes inactive.
- View is a database object, so it occupies the space.
- Without table, view will not work.
- Updation is possible for simple view but not for complex views, they are read only type of views.

Table	View
A table is a database entity that stores data in the form of rows and columns.	A view is a virtual table used to view or manipulate some parts of the table. It also has rows and columns as real ordered tables.
A table stores the data.	A view only extracts data from the table.
A table can only be created or dropped.	A view can be recreated.
A table is an independent database entity.	A view is dependent on the table.
It is stored in physical storage as it occupies real space on systems.	It is not stored physically. It only requires some space in memory whenever we run its query.

A table gives results faster.	A view gives results slower because it has to run its queries each time to retrieve the information from the table.
We can add, delete or update the data in a table.	We cannot modify data from a view. We can change the data in the base

Materialized View

A Materialized View saves the data supplied by the view configuration query and is updated automatically as data in the underpinning tables changes. A view can be “materialized” by storing the tuples of the view in the database.

Materialized view is also a logical virtual table, but in this case the result of the query is stored in the table or the disk. The performance of the materialized view is better than normal view since the data is stored in the disk.

When the results of a view expression are stored in a database system, they are called materialized views. [SQL](#) does not provide any standard way of defining materialized view. The process of keeping the materialized views updated is known as view maintenance. Database system uses one of the three ways to keep the materialized view updated:

Update the materialized view as soon as the relation on which it is defined is updated. Update the materialized view every time the view is accessed. Update the materialized view periodically.

```
SQL> create materialized view mat_std as
```

```
2 select * from std;
```

Materialized view created.

```
SQL> select * from std;
```

```
ROLLNO NAME
```

```
-----
```

```
1 dddd
```

```
2 asas
```

```
3 vbb
```

```
4 bnm
```

```
SQL> select * from mat_std;
```

```
ROLLNO NAME
```

```
-----
```

```
1 dddd
```

```
2 asas
```

```
3 vbb
4 bnm
SQL> insert into std values(5,'Jay');
```

1 row created.

```
SQL> select * from std;
```

ROLLNO NAME

1 dddd

2 asas

3 vbb

4 bnm

5 Jay

```
SQL> select * from mat_std;
```

ROLLNO NAME

1 dddd

2 asas

3 vbb

4 bnm

Read Only view

Read only view did not perform Data manipulation operation like insert,delete,update.From read only view we can only retrieve data .This is apply with read only constraint.

```
SQL> create or replace view vw_std
```

```
2 as select * from std with read only;
```

View created.

Object View

Just as a view is a virtual table, an *object view* is a virtual object table.

Oracle provides object views as an extension of the basic relational view mechanism. By using object views, you can create virtual object tables from data - of either built-in or user-defined types - stored in the columns of relational or object tables in the database.

Object views provide the ability to offer specialized or restricted access to the data and objects in a database. For example, you might use an object view to provide a version of an employee object table that doesn't have attributes containing sensitive data and doesn't have a deletion method.

Object views allow the use of relational data in object-oriented applications. They let users

- Try object-oriented programming techniques without converting existing tables.
- Convert data gradually and transparently from relational tables to object-relational tables.

Use legacy RDBMS data with existing object-oriented applications

Defining Object Views

Conceptually, the procedure for defining an object view is simple:

1. Define an object type to be represented by rows of the object view.
2. Write a query that specifies which data in which relational tables contain the attributes for objects of that type.
3. Specify an object identifier, based on attributes of the underlying data, to allow REFs to the objects (rows) of the object view.

The object identifier corresponds to the unique object identifier that Oracle generates automatically for rows of object tables. In the case of object views, however, the declaration must specify something that is unique in the underlying data (for example, a primary key).

```
SQL> create table std(rollno number primary key,name varchar(10));
```

Table created.

```
SQL> create type std_t as object(
```

```
2 rno number,
```

```
3 n varchar(10));
```

```
4 /
```

Type created.

```
SQL> create view std_view of std_t
```

```
2 with object identifier(rno) as
```

```
3 select * from std;
```

View created.